

PUA 4x4 Mathematical Mapping

Evidentiary proof of coordinates, bit positions, masks and P0/P1 codepoints

MATHEMATICS GATE: PASS

OBSERVED TERMINAL OUTPUT: FAIL - NOT WAIVED

This document proves only the agreed mathematics and its implementation in the current encoder functions. It does **not** certify the generated font, font fallback, terminal shaping, fixed-cell placement, antialiasing or repaint behaviour. The supplied triangle image remains rejected visual evidence.

Gate	Expected	Measured result	Status
Coordinate mathematics	Exact reversible mapping	30,720/30,720 virtual coordinates	PASS
Mask/codepoint bijection	65,536 unique reversible values	65,536 unique; 32,768 per part	PASS
Project mapping code	Match independent oracle	65,536/65,536 masks	PASS
Triangle mask pipeline	No missing or additional virtual pixels	10,860 pixels; 0 missing; 0 additional	PASS
Observed Linux rendering	Connected edge; no cell grid	Detached edge marks and visible seams	FAIL

Decision rule

No further production-font generation is authorized by this document. Mathematics approval is a prerequisite, not a substitute, for later font and runtime evidence.

1. Terms and coordinate systems

A **terminal cell** is one addressable text position. Application coordinates use zero-based column **cx** and row **cy**. ANSI cursor escape sequences use one-based column and row numbers.

A **virtual pixel** is one of sixteen independently selected positions inside a terminal cell. Its global zero-based coordinate is **(vx, vy)**. Its local coordinate inside one cell is **(lx, ly)**, where both values are 0 through 3.

Terminal cells (application coordinates, zero-based)

(0,0)	(1,0)	(2,0)	(3,0)	(4,0)	(5,0)
(0,1)	(1,1)	(2,1)	(3,1)	(4,1)	(5,1)
(0,2)	(1,2)	(2,2)	(3,2)	(4,2)	(5,2)
(0,3)	(1,3)	(2,3)	(3,3)	(4,3)	(5,3)

ANSI cursor: column 4, row 3

One selected terminal cell

3	2	1	0
7	6	5	4
11	10	9	8
15	14	13	12

virtual (13,10) -> local (1,2) -> bit 10

Coordinate conventions

Space	Origin	Horizontal direction	Vertical direction	Range/example
Virtual canvas	top-left	vx increases right	vy increases down	80x24 terminal -> 320x96
Application terminal cells	top-left	cx increases right	cy increases down	cell (3,2)
ANSI cursor	top-left	column increases right	row increases down	column 4, row 3
Local 4x4 grid	cell top-left	lx increases right	ly increases down	local (1,2)
Bit number	MSB-left per row	decreases left-to-right	adds 4 per row	bit 10

The phrase **fedcba9876543210** names bit positions from the numeric most-significant bit to the least-significant bit. It is not a hexadecimal number and is not a string placed directly in Unicode.

2. Authoritative 4x4 bit layout

Every row is MSB-first. Consequently local x values 0,1,2,3 map to descending bit numbers within that row.

local x:		0	1	2	3
local y:	0	bit 3 value 0x0008	bit 2 value 0x0004	bit 1 value 0x0002	bit 0 value 0x0001
	1	bit 7 value 0x0080	bit 6 value 0x0040	bit 5 value 0x0020	bit 4 value 0x0010
	2	bit 11 value 0x0800	bit 10 value 0x0400	bit 9 value 0x0200	bit 8 value 0x0100
	3	bit 15 value 0x8000	bit 14 value 0x4000	bit 13 value 0x2000	bit 12 value 0x1000

```

top row: 3 2 1 0
second row: 7 6 5 4
third row: 11 10 9 8
bottom row: 15 14 13 12

```

```
MSB-to-LSB bit labels: f e d c b a 9 8 7 6 5 4 3 2 1 0
```

Authoritative formula

```

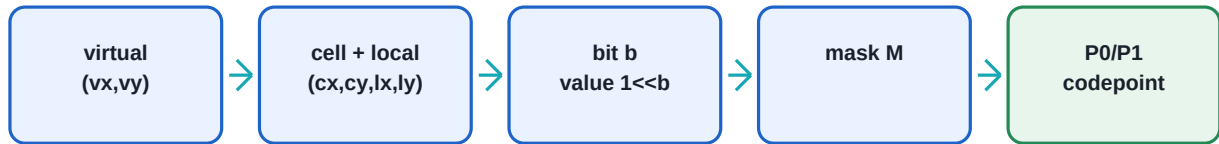
b = 4 * ly + (3 - lx)
bit_value = 1 << b

```

The subtraction 3 - lx is essential. Omitting it would make bit 0 appear at the left, horizontally reflecting every four-pixel row.

3. Forward translation

The forward path begins with one virtual pixel and ends with the Unicode codepoint whose glyph must represent the complete 16-bit cell mask.



Forward translation



Inverse translation must return the identical virtual coordinate

```

cx = vx // 4 cy = vy // 4
ANSI_column = cx + 1 ANSI_row = cy + 1
lx = vx % 4 ly = vy % 4
b = 4 * ly + (3 - lx)
v = 1 << b
M = OR of every v set in the same terminal cell
  
```

Integer division selects the containing terminal cell. Modulo selects the position inside that cell. The bit formula changes only the representation of that local position; it does not change the visual coordinate.

A cell mask is state

One codepoint represents one complete cell state, not one drawing operation. Setting a virtual pixel ORs its bit value into the current mask. Clearing uses AND NOT. Toggling uses XOR. The resulting complete mask selects the replacement codepoint.

4. Worked example: virtual pixel (13,10)

The graphics origin is ANSI row 1, column 1. Virtual and application coordinates remain zero-based; only the cursor escape is one-based.

Terminal cells (application coordinates, zero-based)

(0,0)	(1,0)	(2,0)	(3,0)	(4,0)	(5,0)
(0,1)	(1,1)	(2,1)	(3,1)	(4,1)	(5,1)
(0,2)	(1,2)	(2,2)	(3,2)	(4,2)	(5,2)
(0,3)	(1,3)	(2,3)	(3,3)	(4,3)	(5,3)

ANSI cursor: column 4, row 3

One selected terminal cell

3	2	1	0
7	6	5	4
11	10	9	8
15	14	13	12

virtual (13,10) -> local (1,2) -> bit 10

```

vx = 13, vy = 10
cx = 13 // 4 = 3 cy = 10 // 4 = 2
ANSI column = 3 + 1 = 4 ANSI row = 2 + 1 = 3
lx = 13 % 4 = 1 ly = 10 % 4 = 2
b = 4 * 2 + (3 - 1) = 10
v = 1 << 10 = 1024 = 0x0400

```

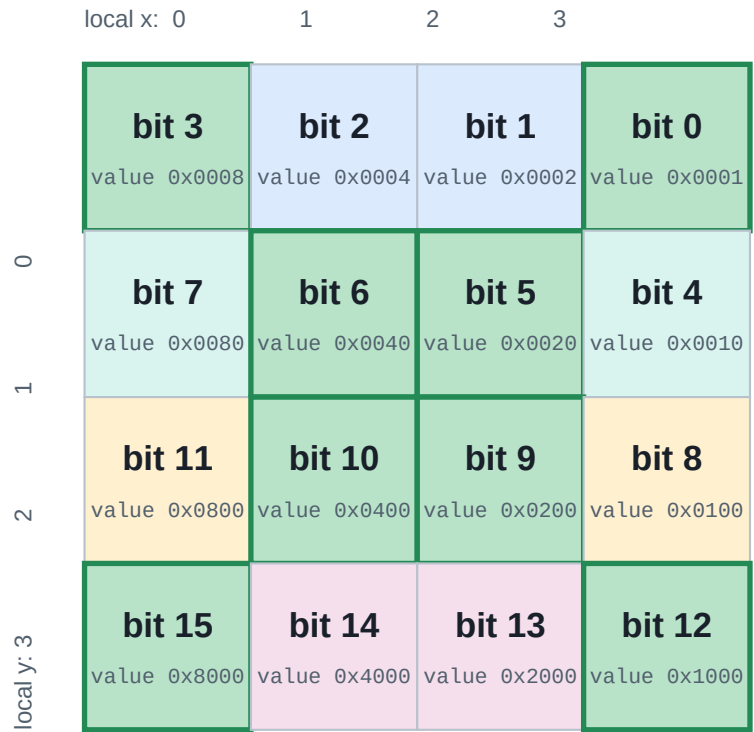
If this is the only selected position, the complete mask is 0x0400. Because 0x0400 is below 0x8000, it belongs to Part 0.

```
codepoint = U+F0000 + 0x0400 = U+F0400
```

Measured reverse result: codepoint U+0F0400 -> mask 0x0400 -> virtual coordinate (13, 10).

5. Multiple selected pixels form one mask

This example selects all four corners and the four central positions. It demonstrates aggregation rather than a one-pixel-only codepoint.



```
selected bits = {15, 12, 10, 9, 6, 5, 3, 0}
M = (1<<15) | (1<<12) | (1<<10) | (1<<9) |
    (1<<6) | (1<<5) | (1<<3) | (1<<0)
M = 0x9669 = binary 1001011001101001
codepoint = U+101669
```

Local row	Selected pattern	Contributing bits	Hex contribution
0	#..#	3 and 0	0x0009
1	##.	6 and 5	0x0060
2	##.	10 and 9	0x0600
3	#..#	15 and 12	0x9000
OR total	#..# / ##. / ##. / #..#	all eight	0x9669

6. P0/P1 codepoint mapping

A 4x4 binary cell has $2^{16} = 65,536$ masks. Each supplementary Private Use Area range used here contains 32,768 assigned slots, so the mapping is split exactly at mask bit 15.

Part 0, $0x0000 \leq M \leq 0x7FFF$:

$cp = 0xF0000 + M$

Part 1, $0x8000 \leq M \leq 0xFFFF$:

$cp = 0x100000 + (M - 0x8000)$

Mask	Part	Calculation	Codepoint	Reverse mask
0x0000	P0	$0xF0000 + 0x0000$	U+0F0000	0x0000
0x0400	P0	$0xF0000 + 0x0400$	U+0F0400	0x0400
0x7FFF	P0	$0xF0000 + 0x7FFF$	U+0F7FFF	0x7FFF
0x8000	P1	$0x100000 + 0$	U+100000	0x8000
0x9669	P1	$0x100000 + 0x1669$	U+101669	0x9669
0xFFFF	P1	$0x100000 + 0x7FFF$	U+107FFF	0xFFFF

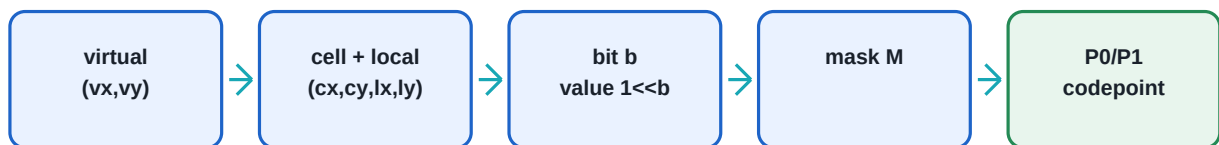
The codepoint is an encoding address. It is not the mask itself in Part 1; the 0x8000 split offset must be removed before adding the P1 base.

7. Reverse translation

The reverse proof starts from a P0 or P1 codepoint, reconstructs the complete mask, locates every set bit and then reconstructs local and global coordinates.

```
P0 reverse: M = cp - 0xF0000
P1 reverse: M = 0x8000 + (cp - 0x100000)

for each set bit b:
  ly = b // 4
  lx = 3 - (b % 4)
  vx = 4 * cx + lx
  vy = 4 * cy + ly
```



Forward translation



Inverse translation must return the identical virtual coordinate

Round-trip identity

```
inverse(forward(vx,vy)) = (vx,vy)
mask(codepoint(M)) = M
mask(local_pixels(M)) = M
```

The terminal cell is required to recover a global virtual coordinate. A glyph codepoint contains the sixteen local on/off states, but it does not encode where that glyph was printed on the terminal.

8. All sixteen one-bit cases

Every local coordinate was translated to one bit, one mask and one codepoint, then reversed to the original local coordinate.

local	bit	value/mask	part	codepoint	reverse local
(0,0)	3	0x0008	P0	U+0F0008	(0,0)
(1,0)	2	0x0004	P0	U+0F0004	(1,0)
(2,0)	1	0x0002	P0	U+0F0002	(2,0)
(3,0)	0	0x0001	P0	U+0F0001	(3,0)
(0,1)	7	0x0080	P0	U+0F0080	(0,1)
(1,1)	6	0x0040	P0	U+0F0040	(1,1)
(2,1)	5	0x0020	P0	U+0F0020	(2,1)
(3,1)	4	0x0010	P0	U+0F0010	(3,1)
(0,2)	11	0x0800	P0	U+0F0800	(0,2)
(1,2)	10	0x0400	P0	U+0F0400	(1,2)
(2,2)	9	0x0200	P0	U+0F0200	(2,2)
(3,2)	8	0x0100	P0	U+0F0100	(3,2)
(0,3)	15	0x8000	P1	U+100000	(0,3)
(1,3)	14	0x4000	P0	U+0F4000	(1,3)
(2,3)	13	0x2000	P0	U+0F2000	(2,3)
(3,3)	12	0x1000	P0	U+0F1000	(3,3)

Result: 16 of 16 cases returned the original local coordinate.

Part 1 first appears at bit 15 because its one-bit value is 0x8000. Bits 0 through 14 are all Part 0 masks.

9. Bitwise state manipulation

The renderer stores a complete 16-bit mask for each terminal cell. Logical operations change exactly one selected bit while preserving all other bits.

Operation	Formula	Meaning	Required invariant
Set	$M' = M \text{ OR } (1 \ll b)$	force bit b to 1	all other bits unchanged
Clear	$M' = M \text{ AND NOT } (1 \ll b)$	force bit b to 0	all other bits unchanged
Toggle	$M' = M \text{ XOR } (1 \ll b)$	invert bit b	all other bits unchanged

```
example bit b = 10, value = 0x0400
set: 0x0000 OR 0x0400 = 0x0400
clear: 0x9669 AND NOT 0x0400 = 0x9269
toggle: 0x9269 XOR 0x0400 = 0x9669
```

The verifier tested set, clear and toggle for every one of 16 bits in every one of 65,536 masks. For each operation it also asserted that the other fifteen bits were unchanged.

3,145,728 Boolean assertions passed.

10. Exhaustive evidence and independence

The test program contains an independent oracle. It does not import the project mapping functions when calculating expected values. Only after the oracle passes are the project functions loaded and compared against it.

Test	Population	Expected	Measured	Status
Local one-bit round trip	16	16 identities	16	PASS
80x24 virtual canvas	320x96	30,720 identities	30,720	PASS
Mask -> codepoint -> mask	65,536	all identities	65,536	PASS
Unique Unicode addresses	65,536	no collisions	65,536	PASS
Part populations	2 parts	32,768 each	32,768 each	PASS
Project mapping comparison	65,536 masks	all match oracle	65,536	PASS
Boolean assertions	all masks and bits	all invariants	3,145,728	PASS
Triangle encode/decode	10,860 pixels	0 missing/additional	0 / 0	PASS

What this establishes

For the agreed convention, the formulas are internally consistent, reversible and collision-free. The current Python mapping functions and triangle mask construction produce the same mathematical masks as the independent oracle.

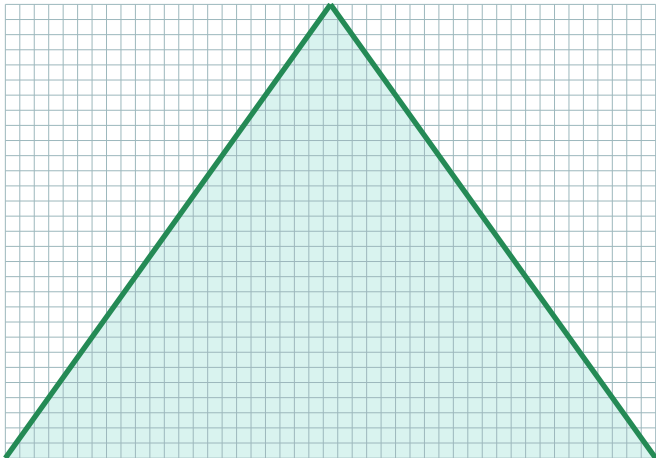
What this does not establish

It does not establish that a particular codepoint is connected to the correct stored outline, that the correct face is selected at runtime, or that the terminal paints that outline correctly inside its fixed cell.

11. Expected triangle result at the mathematics gate

The triangle test builds one analytic set of virtual pixels, encodes those pixels into per-cell masks and independently decodes the masks back to virtual coordinates.

Expected:
one connected
filled set
with no detached
edge pixels



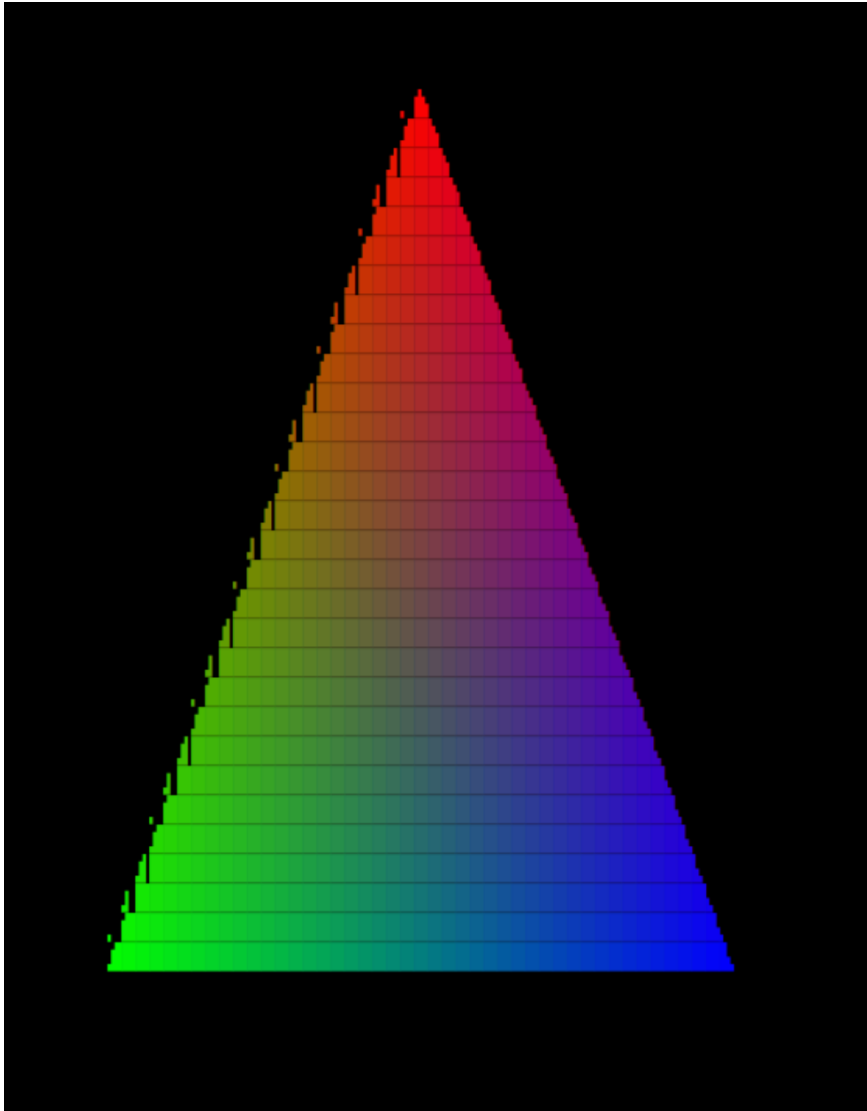
Quantity	Expected	Measured
Analytic triangle virtual pixels	10,860	10,860
Nonempty terminal cells	derived, not assumed	718
Missing after mask decode	0	0
Unexpected after mask decode	0	0

Mathematical outcome: the decoded virtual-pixel set is exactly equal to the original analytic set.

Therefore detached rendered marks are not accepted as part of the intended triangle. They arise after the tested mask/codepoint calculation stage or from an as-yet-unmodelled terminal presentation stage.

12. Real observed terminal outcome

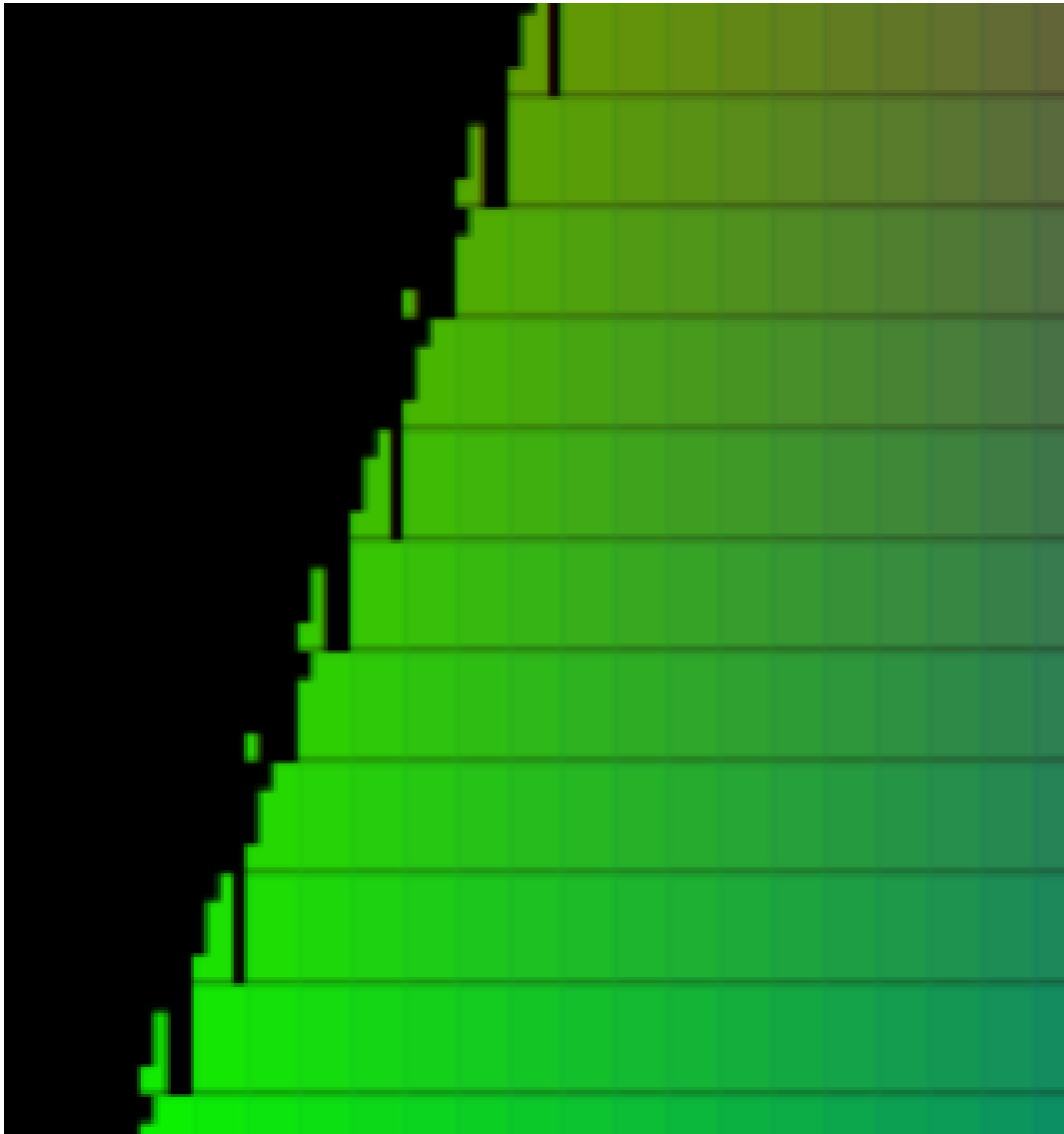
The following image is the user-supplied Linux result from 2026-08-08, using the dedicated PUA 4x4 profile and the triangle demonstration.



Observed result: visible horizontal and vertical cell boundaries and periodic detached/discontinuous marks along the left edge.

Source image SHA-256: 6ff2489bd63fc2ddb655f9d5e8d738589d930bf2a60085c185a7671d6c60e13a

13. Magnified failure evidence



The magnified edge is visually inconsistent with the connected set proved by the mathematical triangle round trip. The image is therefore a failed end-to-end result even though the mathematics gate passes.

Detail image SHA-256: [424ddb744e2ff3eb24b366057a3c804c7baaa77b042f19e6350783fa1359aab6](#)

No inference is made here about which later layer is responsible. Candidate causes to test only after mathematics approval include cmap-to-glyph association, outline orientation, fallback selection, shaping, cell metrics, antialiasing, clipping and repaint behaviour.

14. Evidence gates and stopping rule

Gate	Required evidence	Current result	May advance?
A. Mathematical convention	reviewed formulas and diagrams	awaiting user approval	NO
B. Exhaustive arithmetic	machine evidence for all masks	PASS	only after A
C. Source implementation	independent oracle comparison	PASS	only after A
D. Font cmap	every codepoint links to expected mask	not accepted in this document	NO
E. Glyph outline	each bit occupies expected physical quadrant	not accepted in this document	NO
F. Runtime selection	installed face proven for both parts	not accepted in this document	NO
G. Terminal raster	pixel capture equals expected bitmap	FAIL	NO

Stopping rule

Font generation and font-level corrective work remain paused until Gate A is explicitly confirmed. Later gates must be evidenced independently and linked end-to-end; a pass at an earlier gate cannot override visible failure at a later gate.

Required approval question

Is the authoritative layout, forward formula, P0/P1 mapping and inverse formula in this document exactly the intended mathematical convention?

Appendix A. Reproduction and evidence files

Run from the repository root:

```
cd experiments/pua-4x4
python3 verify_mathematical_mapping.py
```

Expected console summary

```
PASS: authoritative 16-position MSB-left table
PASS: 30,720 virtual-coordinate round trips
PASS: 65,536 mask/codepoint/mask round trips
PASS: 65,536 unique codepoints (32,768 per part)
PASS: 3,145,728 OR/AND-NOT/XOR assertions
PASS: project mapping functions match the independent oracle
PASS: triangle masks decode to the same 10,860 virtual pixels
FAIL (recorded, not waived): observed terminal triangle output
```

Artifact	Purpose	SHA-256
pua4x4-mathematics-proof-v1.0.json	machine-readable complete proof	14c85d95556723bb3ac870301046d62c a348ae96bf01ea5a0336d244b0937011
pua4x4-one-bit-table-v1.0.csv	all sixteen one-bit cases	391fcd9203a076eacb7f66f3b4d65d10 8ed09ff37f4c56ca1644770e2319669d
observed-triangle-2026-08-08-full.png	full observed failure	6ff2489bd63fc2ddb655f9d5e8d73858 9d930bf2a60085c185a7671d6c60e13a
observed-triangle-2026-08-08-detail.png	magnified observed failure	424ddb744e2ff3eb24b366057a3c804c 7baaa77b042f19e6350783fa1359aab6

Document conclusion

The agreed mathematics is exhaustively self-consistent and matches the current Python address/mask/codepoint implementation. The current terminal rendering is visibly wrong and remains an unresolved failed gate. Approval of this document authorizes only progression to font-level evidence, not acceptance of the existing font.